CS3400 Project Defense Guide

A practical walkthrough for explaining the design, implementation, tradeoffs, and remaining work

Project: Campus Transport Resource Management System

Purpose: Help each student explain their assigned module clearly and defend the team design accurately.

How to use this guide

Start with the short system overview, then prepare the section for your assigned student role. Use the sample answers as a base, but explain the actual code in your own words. The limitations section is part of a credible defense: do not claim a feature is complete if the program does not demonstrate it.

1. System overview

The system manages campus transport vehicles, requests, and a network of campus locations. The application is written in C++ and uses manually implemented data structures to demonstrate the course concepts.

A concise opening statement

> “We built a C++ campus transport management system around three modules. Student A manages vehicle resources in a manually implemented linked list. Student B manages requests using a FIFO queue, a cancellation stack, and a priority heap. Student C provides a search tree, a hash table, and a weighted graph for campus navigation. The main program connects the modules and stores vehicle, request, and map data in separate text files.”

At startup, the program loads vehicles from **vehicles.txt**, pending requests from **requests.txt**, and the campus graph from **data/locations.txt**. At exit, it saves the application data. Each structure is chosen for the operations it supports; the project is not using one structure for every task.

Current top-level menu

Choice	Feature	Primary owner
1	Resource management	Student A
2	Search resources by ID	Student A
3	Sort resources	Student A
4	Requests and activities	Student B
5	Priority request management	Student B
6	Undo last request cancellation	Student B
7	Tree operations	Student C
8	Hash table lookup	Student C
9	Campus navigation	Student C
10	System analysis	Shared / Student C implementation

2. Student A - Resource management

Responsibility: model and manage vehicle resources, use dynamic storage, perform resource operations, and support searching, sorting, and file persistence.

How the implementation works

The **Vehicle** class groups a vehicle ID, plate, category, type, route, capacity, and status into one record. **Vehiclelist** stores records in singly linked nodes. Each node contains a Vehicle and a pointer to the next node. The list tracks its head and tail.

The resource menu supports adding, removing, updating, and displaying vehicles. Search is available by linear traversal or binary search. Sorting is available by ID, capacity, type, or plate using insertion sort or merge sort. The vehicle list is loaded from and saved to **vehicles.txt**.

Defense answer: why a linked list?

> “The number of records can change while the program runs. A linked list grows by allocating nodes, so it does not need a fixed maximum-size array. We can insert at the front or end efficiently. The tradeoff is that searching and reaching a position require traversal.”

Complexity to defend

Operation	Time	Reason
Insert at front	O(1)	Update head pointer; no traversal.
Insert at end	O(1)	Tail pointer gives direct access to last node.
Search by ID	O(n)	May inspect every node.
Remove by ID	O(n)	Find node by traversal, then unlink it in O(1).
Linear search	O(n)	Sequential comparison.
Binary search step	O(log n)	Halves a sorted array search range.
Insertion sort	$O(n^2)$ average/worst	Useful for small or nearly sorted data.
Merge sort	O(n log n)	Uses additional O(n) array storage.

Important binary-search detail

Binary search requires sorted data. In this program, the list is copied into a temporary dynamic array and merge-sorted by ID before binary search. Therefore, the complete menu operation takes O(n log n) to prepare the array plus O(log n) for the search; describing the whole operation as O(log n) would omit the sorting cost. The linked list itself remains the main resource store.

Known gap

> The project brief calls for insertion at a specified position. The current list supports insertion at the beginning and end, but not arbitrary-position insertion. A truthful defense is: “We implemented front and end insertion, traversal, search, and deletion. Arbitrary-position insertion remains to be completed; it would walk to the node before the target position, relink pointers, and update the tail if needed.”

Student A: likely questions

Why not use an array? A dynamic array gives indexed access and works well for sorting, but a linked list demonstrates dynamic node allocation and supports efficient insertion at the head and tail. The tradeoff is slower lookup and no direct index access.

Why does end insertion take O(1)? The list stores a tail pointer, so it can connect the new node directly rather than traversing from the head.

Why sort a copy? Sorting a temporary array lets binary search use contiguous indexed storage without changing the resource list's stored order.

What is the difference between insertion sort and merge sort? Insertion sort is simple and can be good for small or nearly sorted data, but has quadratic average and worst-case time. Merge sort has O(n log n) time and needs extra memory.

3. Student B - Requests and activities

Responsibility: manage normal requests in arrival order, prioritize urgent requests, process and cancel requests, provide cancellation undo, and schedule activities.

Request records

The **Request** class stores the request ID, user, registered vehicle ID, pickup, destination, time, status, and priority. This keeps related request data together while allowing the queue, stack, and heap to store the same record type.

Normal requests: linked FIFO queue

FIFO means first in, first out. A linked queue tracks its front and rear. New requests are added at the rear, and processing removes the front request. The first normal request submitted is therefore the first normal request processed.

> "FIFO fits ordinary requests because it preserves arrival order and gives users a predictable processing rule. Enqueue and dequeue are O(1) because the queue stores both front and rear pointers."

Urgent requests: array-backed binary min-heap

Priority 1 is Emergency, 2 is High, 3 is Normal, and 4 is Low. The heap is a min-heap by priority number: smaller numbers represent greater urgency. Equal priorities are ordered by request ID. Insertion bubbles an item upward; removing the root moves the last item down until the heap property is restored.

For a heap entry at array index i, its parent is (i - 1) / 2, its left child is 2i + 1, and its right child is 2i + 2. The priority queue is the behavior; the heap is the implementation.

Cancellation undo: linked stack

A stack is LIFO: last in, first out. Cancelling a request pushes it onto the stack. Undo pops the most recently cancelled request and restores it to its pending structure. Push and pop are O(1). The undo stack is session-only and is not saved to disk.

Activities and persistence

Activities are represented using Request records. Their title, venue, date/time, and priority are placed in the request fields; priority decides whether the activity enters the normal queue or urgent heap. This reuses the request pipeline but is less expressive than a dedicated Activity class.

Pending requests are stored as pipe-delimited lines in **requests.txt**. The file does not preserve processed-request history or cancellation undo history. A pipe character inside a field would also break the simple file parser.

Complexity to defend

Operation	Time	Reason
Queue enqueue/dequeue	O(1)	Rear insertion and front removal.
Stack push/pop	O(1)	Operate at top pointer.
Heap insert/remove top	O(log n)	Bubble up/down through heap height.
Heap peek	O(1)	Most urgent item is at root.
Find request by ID in heap	O(n)	Heap is ordered by priority, not ID.
Display pending entries	O(n)	Every record must be visited.

Student B: critical design limitation

Normal requests and urgent requests are stored in separate structures and have separate process actions. That means a user can process a normal FIFO request while an urgent request is still waiting in the heap. The implementation demonstrates both FIFO and priority processing, but does not enforce one global ordering across every pending request.

Say this accurately in a defense

> “We maintain a FIFO queue for ordinary requests and a priority heap for urgent requests. They are processed through separate menu actions. A unified process-next policy would be needed if the requirement means an urgent request must always block every lower-priority request globally.”

Student B: likely questions

Why use a heap? It keeps the most urgent request at the root and supports insertion and removal in logarithmic time.

Why is arbitrary ID lookup O(n) in the heap? The heap property orders by priority, not by request ID. It cannot use binary search by ID.

What is the priority-queue/heap relationship? A priority queue defines the required behavior; a heap is one efficient way to implement that behavior.

What happens when the program exits? Pending requests are saved. The current session’s cancellation stack and processed-request history are not persisted.

4. Student C - Trees, hashing, navigation

Responsibility: organize and retrieve vehicle records using a tree and hash table, model the campus network as a graph, and implement graph traversal and connectivity/path analysis.

Binary search tree

The vehicle BST is ordered by vehicle ID. Smaller keys go left and larger keys go right. It supports insertion, search, deletion, inorder/preorder/postorder and level-order traversals, plus size and height. Inorder traversal produces IDs in sorted order.

Search, insertion, and deletion are typically O(log n) when the tree is reasonably balanced, but O(n) in the worst case. Sorted insertion can form a chain because this is not a self-balancing tree.

Hash table with separate chaining

The hash table maps a vehicle ID to a vehicle. A hash function chooses a bucket. If more than one ID maps to that bucket, a linked chain stores the colliding entries. Lookup and insertion are average O(1) when hashing distributes keys well and the load factor is controlled; worst-case operations can reach O(n) if many items land in one chain.

Weighted graph

The campus map uses an adjacency list. Stops are vertices; route connections are edges; edge weights represent distance. The graph supports BFS, DFS, reachability, connected components, shortest path, and load/save using **data/locations.txt**. BFS and DFS with adjacency lists run in O(V + E), where V is the number of stops and E is the number of connections.

> Why weighted edges?

Student C: integration limitation

At startup, the main program copies vehicles into the tree and hash table. Resource Management updates both indexes when a vehicle is added, removed, or updated. However, inserting or deleting directly through the tree or hash menus changes only that structure; it does not update the main vehicle list or vehicles.txt. A robust architecture would centralize mutations in a resource service and rebuild or update all indexes consistently.

Student C: likely questions

Why keep both a BST and a hash table? They demonstrate different access patterns. Hashing supports fast average lookup by ID; the BST maintains key order and supports ordered traversal.

What if IDs are inserted in sorted order? The unbalanced BST may become chain-shaped, making its height and search time O(n). A balanced tree would address that risk.

Why use a graph? Campus stops are connected by routes, so a graph models both connectivity and distance-aware navigation. BFS/DFS answer traversal questions; shortest-path analysis uses the route weights.

5. Team-wide defense and readiness

Explain the structure choice, operation, complexity, and tradeoff together. Avoid saying a structure is efficient without naming the operation and assumptions.

A useful response pattern

> "We chose [structure] because [operation/rule]. The operation works by [brief mechanics]. Its time cost is [complexity] under [assumption]. The tradeoff is [limitation]. An alternative would be [alternative], which would change [tradeoff]."

Requirements that need follow-up

- **Student A:** implement and demonstrate insertion at an arbitrary linked-list position, which the brief lists separately from front/end insertion.
- **Student B:** decide whether priority is required globally. Current normal and urgent processing paths are separate, so an urgent request does not automatically block normal processing.
- **Student B:** consider whether processed history should be saved; currently only pending requests are saved and the undo stack is temporary.
- **Student C:** decide whether direct tree/hash menu edits should update the master vehicle list or be described as structure demonstrations only.
- **Whole team:** run the Student C test driver and add representative tests for resources, requests, queue/stack boundaries, heap priorities, persistence, and invalid input.
- **Whole team:** complete the report with at least five operation complexity analyses and best/average/worst-case discussion where applicable.

Demonstration checklist

- Add a vehicle and show it in display/search; restart and confirm it reloads.
- Show linear search and binary search, explaining why binary search needs sorted data.
- Show insertion sort and merge sort and compare their complexity.
- Submit normal requests in order and process them FIFO.
- Submit emergency and high requests; view and process the heap root.
- Cancel a pending request, undo it, and confirm it is pending again.
- Schedule an activity and explain which request structure receives it.

- Use the tree traversals, hash lookup, and graph BFS/DFS/shortest path.
- Demonstrate empty queue/stack behavior and a missing ID or invalid menu input.

Be honest about incomplete features

> “That feature is not implemented yet. The current version supports [what is present]. To complete it, I would [specific next step], with [complexity/design impact].”

This answer demonstrates that you understand the code and the remaining design work. It is more defensible than claiming full compliance when the program does not yet demonstrate the behavior.

Build verification

The integrated source was built with the repository Makefile using MinGW and C++11. That confirms the modules compile and link together. It does not replace module-level behavioral tests or the test cases required for the final submission.